PROJECT II REPORT

Yamini Buyya – 801482615

Sriya Madhuri Popuri – 801472588

Lavannya Patil – 801471484

1. Introduction

1.1 Project Overview

The *Healthcare EMR System* is a full-stack electronic medical records application designed to streamline and digitize clinical and administrative operations within a healthcare environment. Developed using a modern technology stack—React for the user interface and Node.js with Express and MySQL for backend and data management—the system supports core healthcare workflows such as patient registration, appointment scheduling, prescription management, and real-time audit logging.

This project seeks to address the limitations of paper-based medical records, including data loss, inefficiency, and lack of accessibility. By providing a centralized and secure EMR platform, the system improves the accuracy, consistency, and traceability of healthcare information. It also enhances operational efficiency for staff such as receptionists, nurses, and physicians by enabling controlled access to clinical data based on user roles and permissions. Overall, the system promotes better data integrity, greater accountability, and a more seamless interaction between medical staff and patient information.

1.2 Database Design

The underlying database for the Healthcare EMR System is built using MySQL 8.0 and follows a fully normalized relational structure. The schema is designed to capture essential healthcare operations through a collection of well-defined tables representing key entities such as **Users**, **Roles**, **Patients**, **Doctors**, **Appointments**, **Prescriptions**, **Medications**, and **Audit_Log**.

Referential integrity is maintained through appropriately defined primary and foreign key constraints, ensuring consistent and reliable relationships between entities. To further support system functionality, the database incorporates:

- **Stored procedures** to encapsulate business logic and reduce duplication in backend code
- **Views** to simplify complex queries and support secure, role-restricted data retrieval
- **Indexing strategies** to improve performance for frequently accessed attributes
- **Triggers** to automate audit logging and maintain accountability for critical operations

Together, these enhancements create a scalable, secure, and maintainable database architecture suitable for healthcare workflows.

1.3 Updates from Project 1

Project 1 primarily focused on constructing the foundational database structure and populating it with initial sample data using CREATE and INSERT statements. In Project 2, the system was significantly expanded to support more advanced database capabilities and backend integration. Key updates include:

- **Introduction of Role-Based Access Control (RBAC)** through new relational tables such as
  - *Roles*
  - *Permissions*
  - *Role_Permission*
  - *User_Role*
- **Addition of the Audit_Log table** to support comprehensive audit trails for critical system actions
- **Development of stored procedures** for tasks such as appointment creation, user authentication support, and audit tracking
- **Implementation of database triggers** to automatically record user operations and maintain data consistency
- **Creation of system views**, including vw_user_permissions, to simplify permission retrieval and enhance backend logic
- **Addition of indexes** to optimize query performance and improve scalability

These enhancements collectively elevate the database from a basic relational structure to a fully functional, production-ready EMR backend capable of supporting secure, performant, and accountable healthcare data operations.

2. Functional Requirements

2.1Entities in the Database System

The Healthcare EMR System database includes the following entities:

- **Users**
- **Roles**
- **Permissions**
- **User_Role**
- **Role_Permission**
- **Patients**
- **Doctors**
- **Appointments**
- **Prescriptions**
- **Medications**
- **Prescription_Medication**
- **Audit_Log**
- **Specializations**
- **Departments**
- **Login_History**
- **Notifications**

2.2 Description of New Entities (Introduced in This Project)

The following entities were added or enhanced in Project 2 to support advanced functionality such as RBAC (Role-Based Access Control) and system auditing:

1. Roles

Defines user roles within the system (e.g., Admin, Physician, Receptionist).
Used for permission assignment and access restriction.

2. Permissions

Represents individual system-level actions (e.g., “create_patient”, “view_prescription”).
These granular permissions enable fine-grained access control.

3. Role_Permission

A mapping table connecting roles to permissions.
Determines what actions each role is allowed to perform.

4. User_Role

Maps users to their respective roles.
Supports multi-role assignments for flexibility.

5. Audit_Log

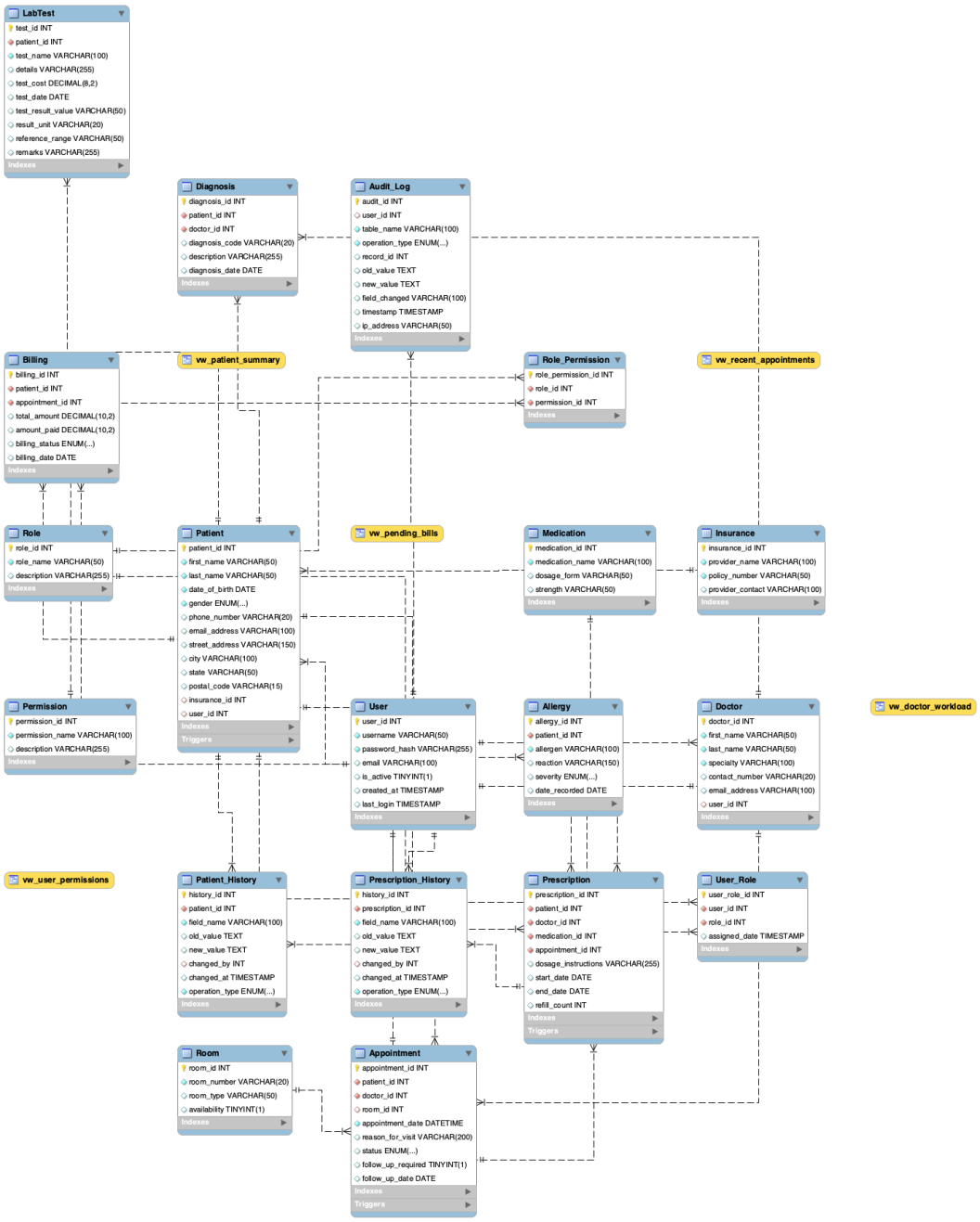
Captures every significant action performed in the system, including user ID, action type, timestamp, and affected record.
This provides traceability and accountability for all critical operations.

6. Login_History

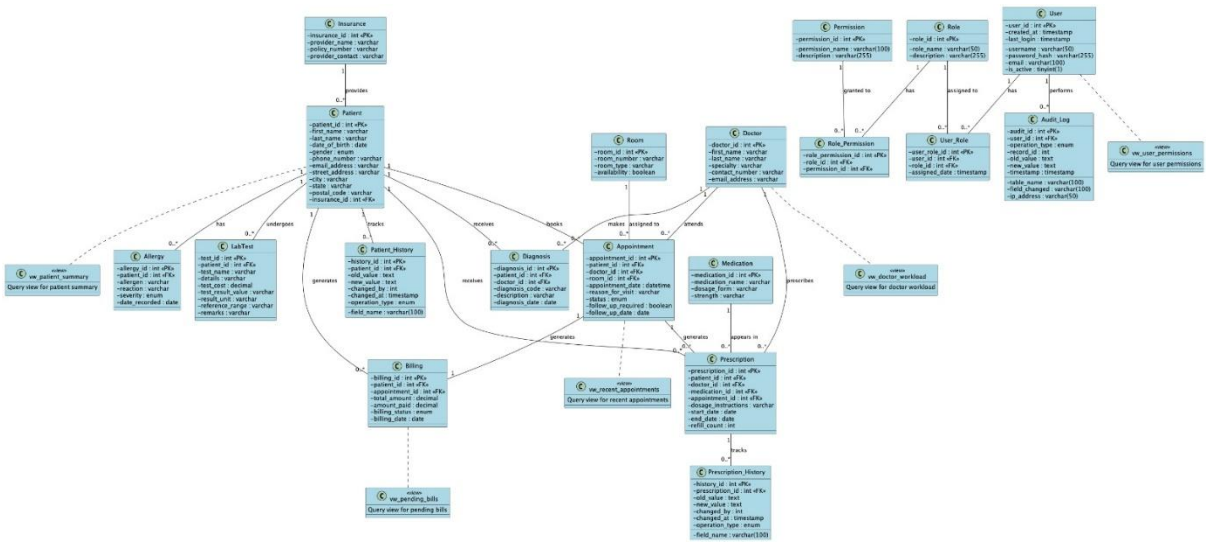
Logs user login attempts, timestamps, and status (success/failure).
Supports security monitoring and anomaly detection.

3. Entity Relationship

3.1 ER diagram



3.2 UML diagram



4. Proof of BCNF Compliance

The database schema used in this project was designed following normalization rules to ensure data integrity, minimize redundancy, and maintain consistency across the system. The schema was normalized up to **Boyce-Codd Normal Form (BCNF)**.

A relation is in **BCNF** if:

- For every functional dependency **X → Y**,
- **X** is a **superkey** of the table.

During the design, functional dependencies were analyzed to ensure each table meets this rule.

Key Justifications

- **User**, **Patient**, **Doctor**, **Medication**, and **Appointment** tables use a **single primary key**, ensuring each non-key attribute depends entirely on the key.
- Linking tables such as **User_Role** and **Role_Permission** use **composite keys**, ensuring no partial or transitive dependency exists.
- **Audit_Log** stores event data where the **primary key uniquely identifies each log entry**, eliminating the possibility of repeated or dependent attributes.
- The **Prescription** table separates prescription details from patient, medication, and doctor records, preventing redundancy and keeping it in BCNF.

Since no table contains:

- Attributes dependent on non-key attributes (no transitive dependencies),
- Composite keys with partial dependencies,
- Violations where a non-candidate key determines another attribute,

The schema satisfies all requirements for **BCNF**.

5. Table Information

Table Name	Description	Attributes	Notes
User	Stores user login and identity information	user_id (PK), username, password_hash, email, created_at	—
Role	Defines system roles	role_id (PK), role_name	—
User_Role	Maps users to roles	user_id (FK), role_id (FK), PRIMARY KEY(user_id, role_id)	Linking table
Permission	Defines permissions in the system	permission_id (PK), permission_name	—
Role_Permission	Maps roles to permissions	role_id (FK), permission_id (FK), PRIMARY KEY(role_id, permission_id)	Linking table
Patient	Stores patient demographic and health details	patient_id (PK), first_name, last_name, dob, gender, contact_info, address, created_at	—
Doctor	Stores physician details	doctor_id (PK), name, specialization, contact_info	—
Appointment	Tracks scheduled appointments	appointment_id (PK), patient_id (FK), doctor_id (FK), appointment_date, status	—
Medication	Medication reference table	medication_id (PK), medication_name, dosage_form	—
Prescription	Stores prescriptions issued to patients	prescription_id (PK), patient_id (FK), doctor_id (FK), medication_id (FK), dosage, created_at	—
Audit_Log	Tracks system-level changes and user activity	log_id (PK), user_id (FK), action, table_name, timestamp	New table
vw_user_permissions (View)	Generated view combining roles and permissions	user_id, role_name, permission_name	Logical view, not a physical table

6. Application Programming Interface (API) Implementation

6.1 Stored Procedures

Definition & Purpose

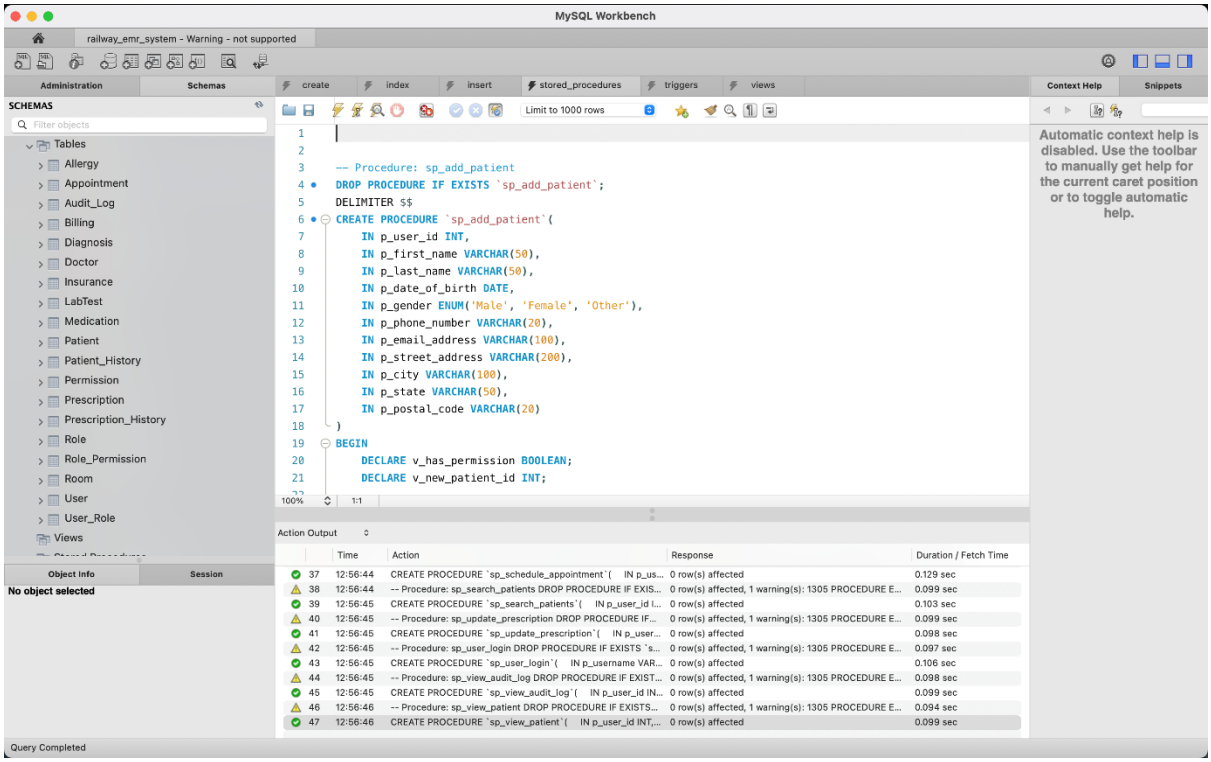
Stored procedures are predefined SQL blocks stored in the database that execute specific tasks such as inserting data, retrieving records, or updating information. They help improve performance, maintain consistency, and simplify application logic.

How Stored Procedures Work in the System

In this system, stored procedures automate repeated operations such as adding employees, retrieving department-wise reports, and updating sensitive information safely. The application calls stored procedures instead of running direct SQL queries, ensuring security and efficiency.

List of Stored Procedures

Procedure Name	Category	Purpose	Key Actions Performed
sp_add_patient	Patient Management	Add a new patient	Checks permission → inserts patient → returns new patient ID
sp_update_patient	Patient Management	Update patient details	Validates permission → updates patient fields → audit via trigger
sp_view_patient	Patient Management	View a specific patient	Checks VIEW permission → returns patient details (partial code provided)
sp_list_patients	Patient Listing	Fetch paginated list of patients	Logs SELECT in Audit_Log → returns patients (limit + offset)
sp_search_patients	Patient Search	Search by name	Logs SELECT → searches by first/last name → returns first 50
sp_schedule_appointment	Appointment Management	Create/schedule appointment	Inserts new appointment → returns appointment ID + message
sp_get_doctor_appointments	Appointment Listing	Fetch appointments for a doctor	Identifies doctor from username → returns list of appointments
sp_add_prescription	Prescription Management	Add a prescription for a patient	Checks permission → validates appointment → inserts → logs INSERT
sp_update_prescription	Prescription Management	Update prescription details	Checks permission → updates fields → logs UPDATE
sp_delete_prescription	Prescription Management	Delete a prescription	Checks permission → verifies record exists → deletes securely
sp_user_login	Authentication	Login & verify user	Validates hash, updates last_login, returns doctor details if applicable
sp_check_permission	RBAC (Access Control)	Permission validation	Checks if user has the required permission via role mappings
sp_view_audit_log	Auditing	View system logs	Logs access to Audit_Log → returns recent audit entries



6.2 User Authentication

Definition & Purpose

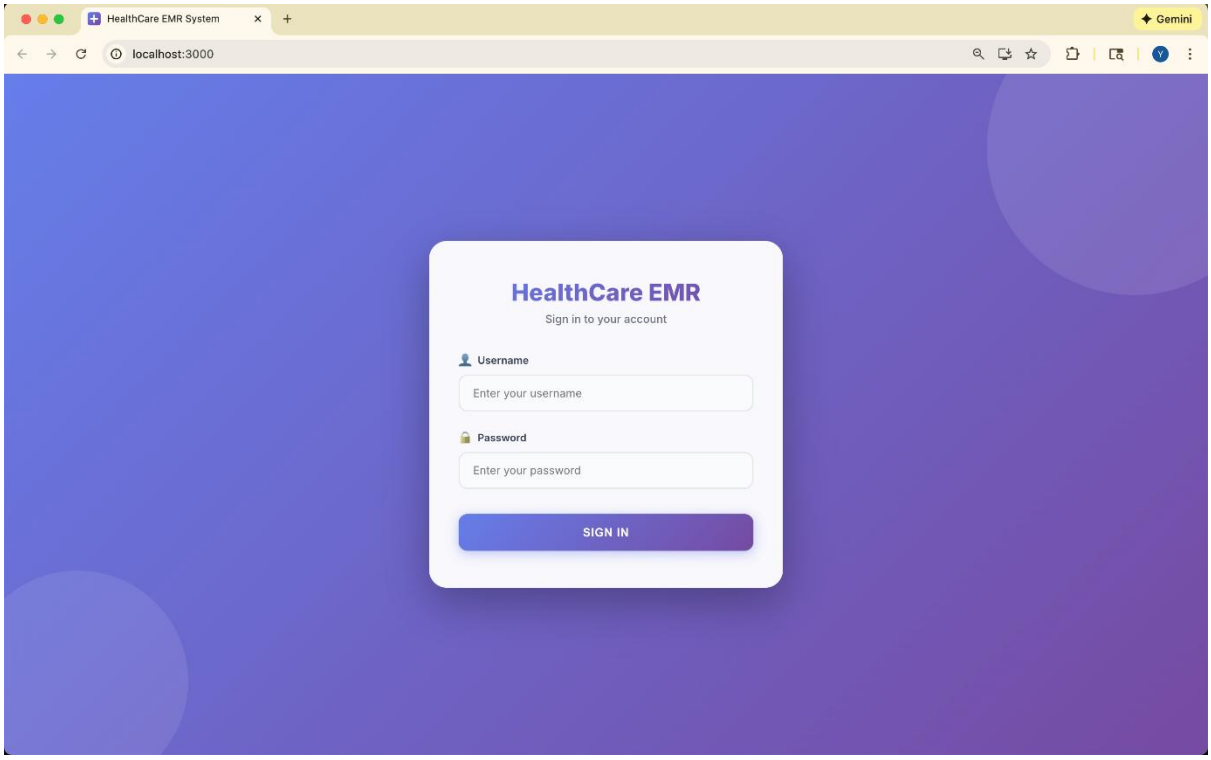
User authentication verifies the identity of users attempting to access the system. It prevents unauthorized access and ensures data security.

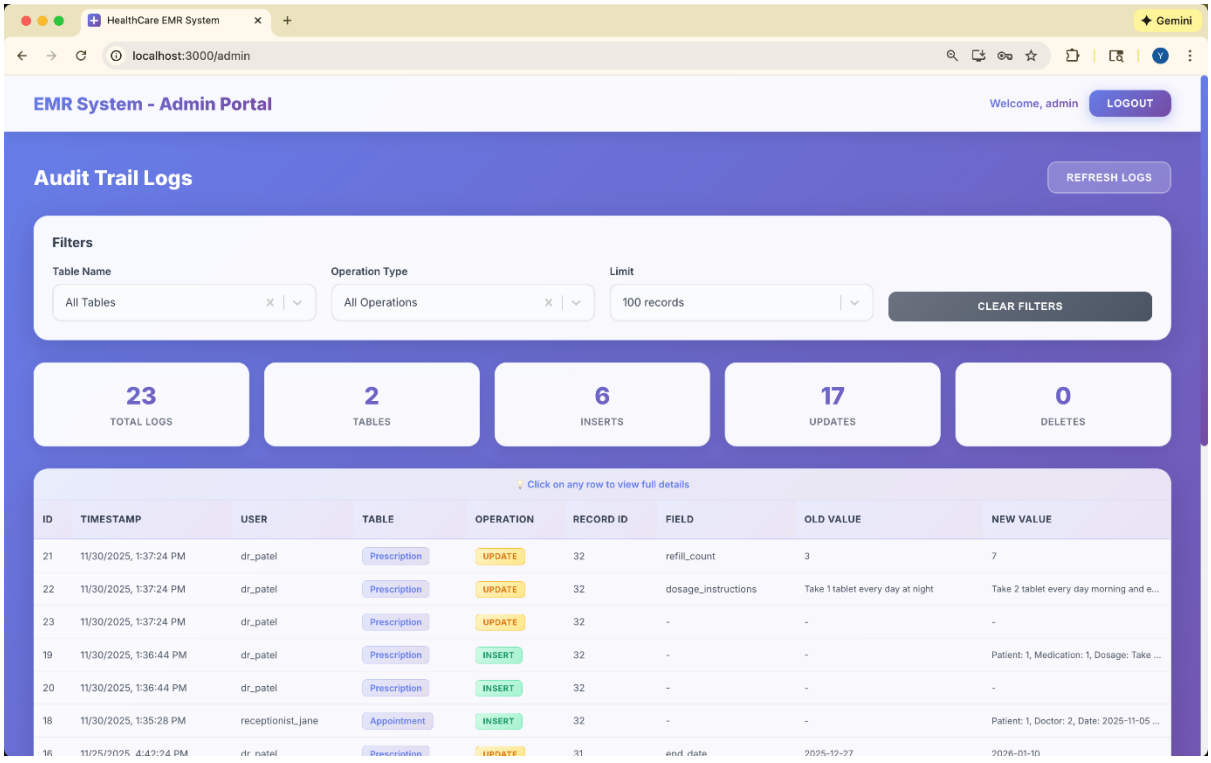
How Authentication Works

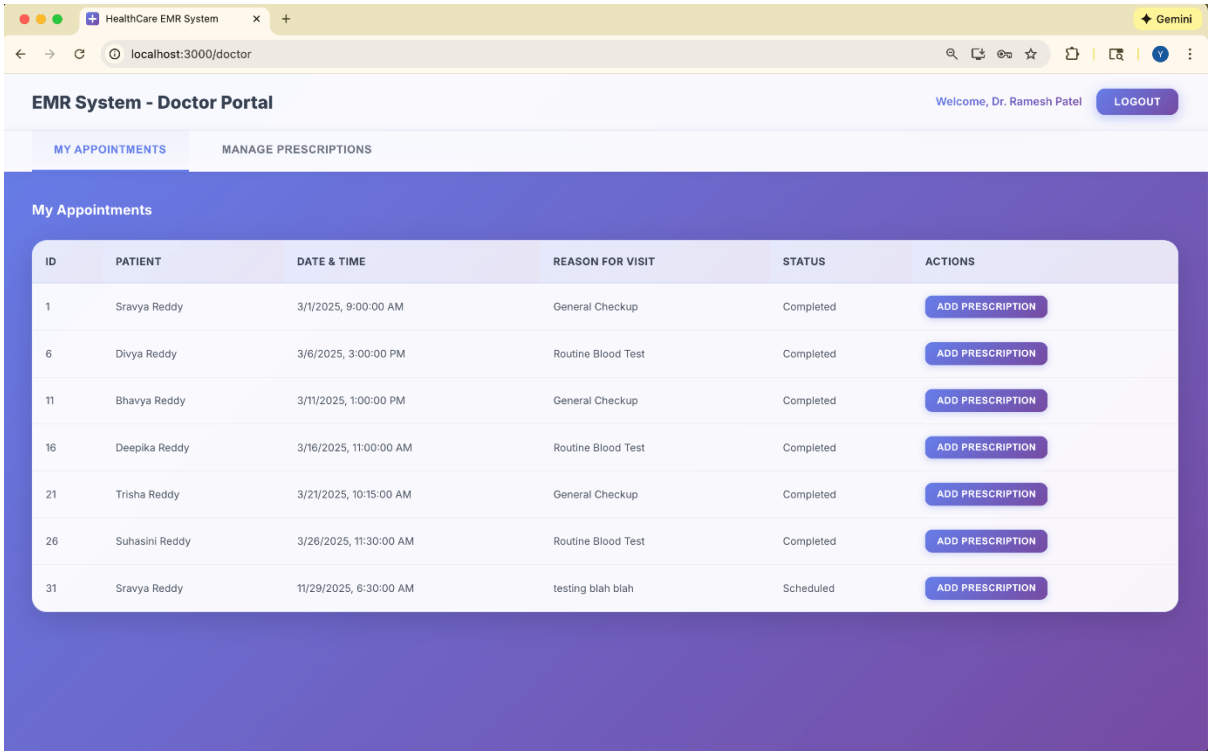
Users must log in with a valid username and hashed password. Permissions determine which operations they can perform.

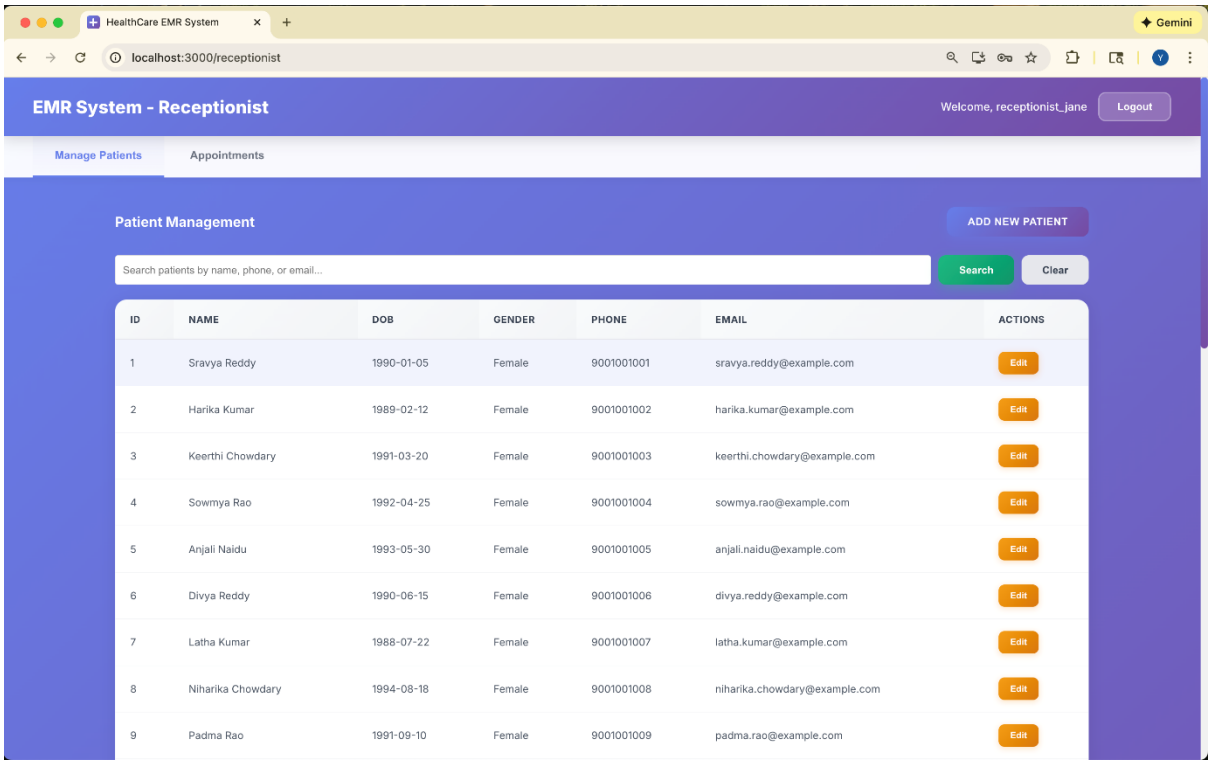
User List & Privileges

Username	Role	Privileges
admin	**Admin**	Full system access: manage users, assign roles, view all records, CRUD on all tables, audit logs, system configuration.
doctor	**Doctor**	Access to patient records, create/update prescriptions, view appointments, update appointment notes, limited read access to billing.
receptionist	**Receptionist**	Manage appointment scheduling, update patient details, check patient visit history, no access to prescriptions.


HealthCare EMR System
Gemini
localhost:3000
HealthCare EMR
Sign in to your account
Username
Enter your username
Password
Enter your password
SIGN IN


HealthCare EMR System
Gemini
localhost:3000/admin
EMR System - Admin Portal
Welcome, admin
LOGOUT
Audit Trail Logs
REFRESH LOGS
Filters
Table Name
All Tables
Operation Type
All Operations
Limit
100 records
CLEAR FILTERS
23
TOTAL LOGS
2
TABLES
6
INSERTS
17
UPDATES
0
DELETES
Click on any row to view full details
ID
TIMESTAMP
USER
TABLE
OPERATION
RECORD ID
FIELD
OLD VALUE
NEW VALUE
21
11/30/2025, 1:37:24 PM
dr_patel
Prescription
UPDATE
32
refill_count
3
7
22
11/30/2025, 1:37:24 PM
dr_patel
Prescription
UPDATE
32
dosage_instructions
Take 1 tablet every day at night
Take 2 tablet every day morning and e...
23
11/30/2025, 1:37:24 PM
dr_patel
Prescription
UPDATE
32
-
-
-
19
11/30/2025, 1:36:44 PM
dr_patel
Prescription
INSERT
32
-
-
Patient: 1, Medication: 1, Dosage: Take ...
20
11/30/2025, 1:36:44 PM
dr_patel
Prescription
INSERT
32
-
-
-
18
11/30/2025, 1:35:28 PM
receptionist_jane
Appointment
INSERT
32
-
-
Patient: 1, Doctor: 2, Date: 2025-11-05 ...


HealthCare EMR System
Gemini
localhost:3000/doctor
EMR System - Doctor Portal
Welcome, Dr. Ramesh Patel
LOGOUT
MY APPOINTMENTS
MANAGE PRESCRIPTIONS
My Appointments
ID
PATIENT
DATE & TIME
REASON FOR VISIT
STATUS
ACTIONS
1 Sravya Reddy 3/1/2025, 9:00:00 AM General Checkup Completed ADD PRESCRIPTION
6 Divya Reddy 3/6/2025, 3:00:00 PM Routine Blood Test Completed ADD PRESCRIPTION
11 Bhavya Reddy 3/11/2025, 1:00:00 PM General Checkup Completed ADD PRESCRIPTION
16 Deepika Reddy 3/16/2025, 11:00:00 AM Routine Blood Test Completed ADD PRESCRIPTION
21 Trisha Reddy 3/21/2025, 10:15:00 AM General Checkup Completed ADD PRESCRIPTION
26 Suhasini Reddy 3/26/2025, 11:30:00 AM Routine Blood Test Completed ADD PRESCRIPTION
31 Sravya Reddy 11/29/2025, 6:30:00 AM testing blah blah Scheduled ADD PRESCRIPTION


HealthCare EMR System
Gemini
localhost:3000/receptionist
EMR System - Receptionist
Welcome, receptionist_jane
Logout
Manage Patients
Appointments
Patient Management
ADD NEW PATIENT
Search patients by name, phone, or email...
Search
Clear
ID
NAME
DOB
GENDER
PHONE
EMAIL
ACTIONS
1 Sravya Reddy 1990-01-05 Female 9001001001 sravya.reddy@example.com Edit
2 Harika Kumar 1989-02-12 Female 9001001002 harika.kumar@example.com Edit
3 Keerthi Chowdary 1991-03-20 Female 9001001003 keerthi.chowdary@example.com Edit
4 Sowmya Rao 1992-04-25 Female 9001001004 sowmya.rao@example.com Edit
5 Anjali Naidu 1993-05-30 Female 9001001005 anjali.naidu@example.com Edit
6 Divya Reddy 1990-06-15 Female 9001001006 divya.reddy@example.com Edit
7 Latha Kumar 1988-07-22 Female 9001001007 latha.kumar@example.com Edit
8 Niharika Chowdary 1994-08-18 Female 9001001008 niharika.chowdary@example.com Edit
9 Padma Rao 1991-09-10 Female 9001001009 padma.rao@example.com Edit

6.3 Views

Definition & Purpose

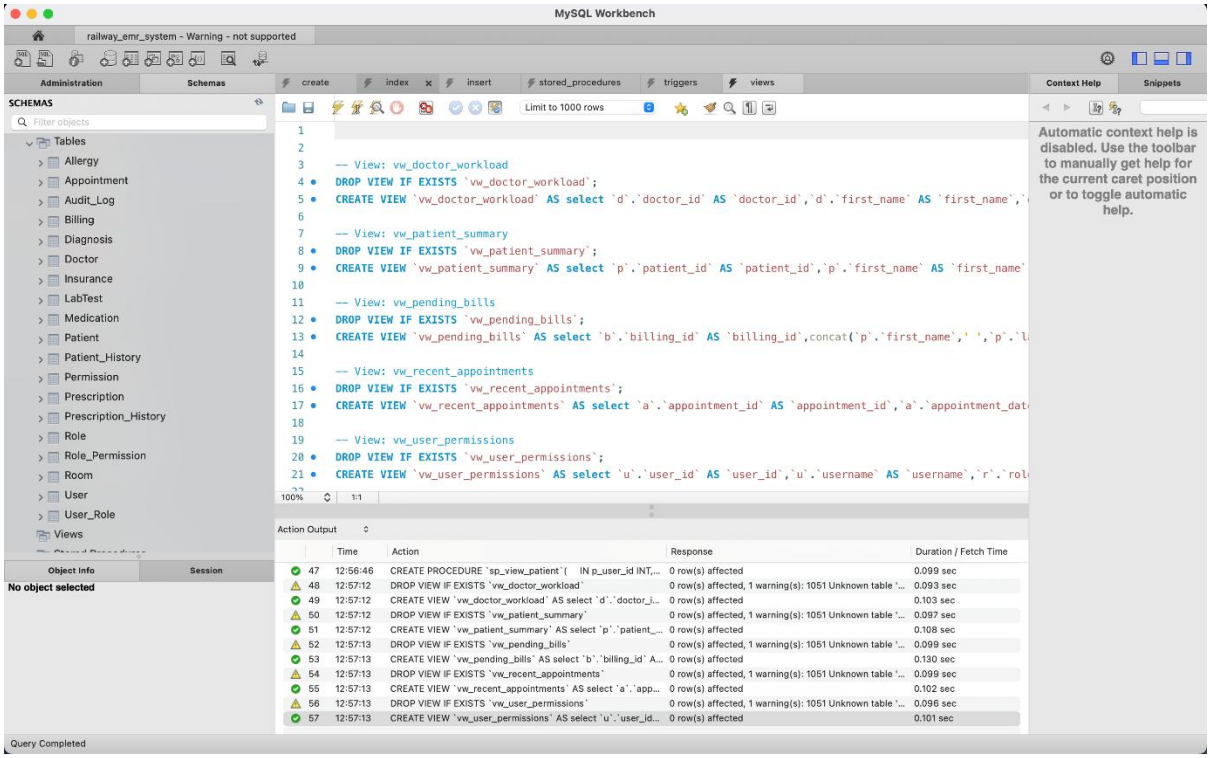
A view is a virtual table created using a query. It simplifies data access, hides sensitive columns, and improves readability.

Views in the System

Views are used for reporting and restricting access to specific data such as employee salary lists or department information.

List of Views

View Name	Purpose / Category	Main Function	Key Data Displayed
vw_doctor_workload	Doctor Performance & Analytics	Summarizes each doctor's workload and revenue	Total appointments, prescriptions, and billing revenue grouped per doctor
vw_patient_summary	Patient Overview	Provides a complete summary of each patient	Age, gender, insurance provider, total appointments, total prescriptions
vw_pending_bills	Financial Tracking	Displays all unpaid or partially paid bills	Patient name, total amount, amount paid, amount due, billing status
vw_recent_appointments	Appointment Monitoring	Shows all recent appointments in last 30 days	Patient name, doctor name, specialty, room number, status, reason for visit
vw_user_permissions	Security & RBAC	Shows user → role → permission mapping	Username, role, and all permissions assigned through RBAC



6.4 Indexes

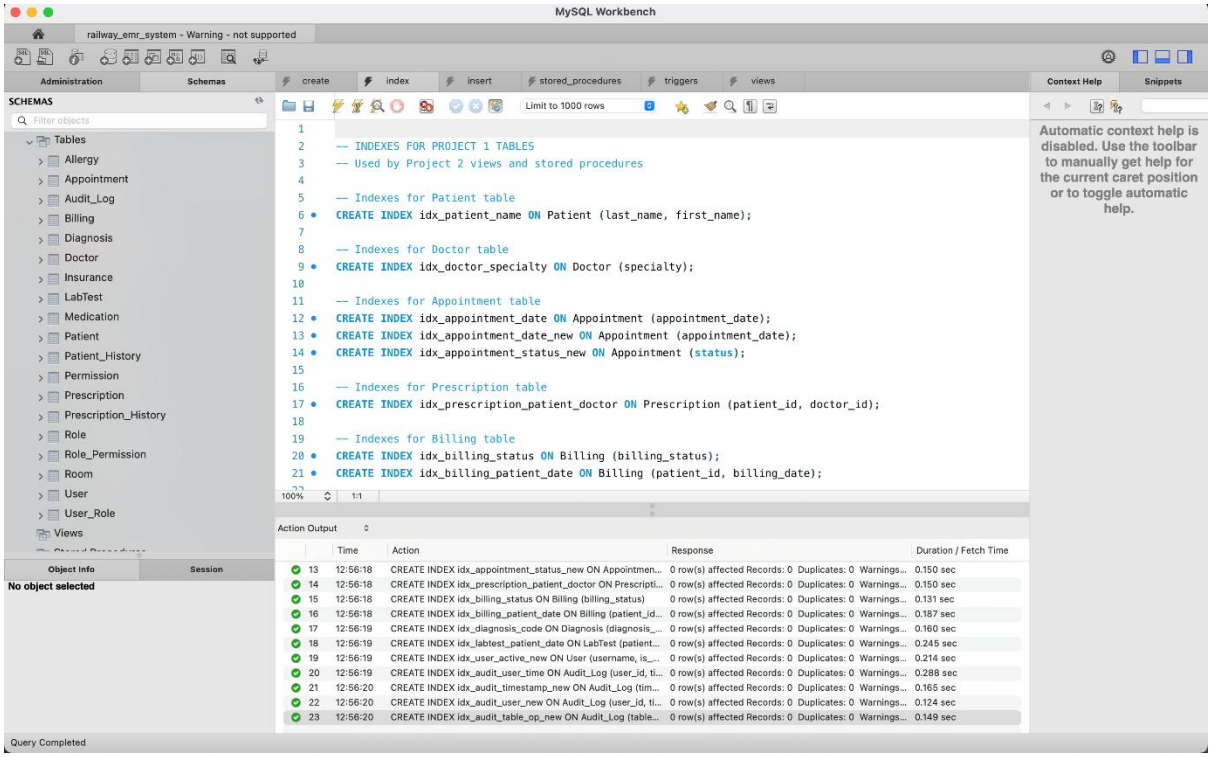
Definition & Purpose

Indexes improve database query performance by enabling faster lookups, especially on frequently accessed columns.

Indexes in the System

Index Name	Table	Columns Indexed	Purpose / Benefit
idx_patient_name	Patient	(last_name, first_name)	Speeds up patient search by name for appointments, billing, and reports.
idx_doctor_specialty	Doctor	(specialty)	Improves filtering doctors by specialties for appointments and workload views.
idx_appointment_date	Appointment	(appointment_date)	Faster retrieval of appointments by date (used in recent appointments view).
idx_appointment_date_new	Appointment	(appointment_date)	Duplicate index — supports same date-based filtering; improves SELECT + ORDER BY.

idx_appointment_status_new	Appointment	(status)	Optimizes queries filtering appointments by status (Completed, Pending, Cancelled).
idx_prescription_patient_doctor	Prescription	(patient_id, doctor_id)	Accelerates queries linking patients & doctors to prescriptions.
idx_billing_status	Billing	(billing_status)	Supports filtering unpaid / pending bills (used in vw_pending_bills).
idx_billing_patient_date	Billing	(patient_id, billing_date)	Optimizes billing history retrieval by patient and date.
idx_diagnosis_code	Diagnosis	(diagnosis_code)	Faster diagnosis lookup by coded value.
idx_labtest_patient_date	LabTest	(patient_id, test_date)	Enables quick access to lab results in chronological order.
idx_user_active_new	User	(username, is_active)	Improves login authentication and active-user checks.
idx_audit_user_time	Audit_Log	(user_id, timestamp)	Speeds up searching audit logs for a specific user over time.
idx_audit_timestamp_new	Audit_Log	(timestamp DESC)	Enables fast retrieval of latest audit logs.
idx_audit_user_new	Audit_Log	(user_id, timestamp DESC)	Combines user filter + recent activity — useful for security review.
idx_audit_table_op_new	Audit_Log	(table_name, operation_type, timestamp DESC)	Helps track operations on specific tables for compliance audits.



6.5 Role-Based Access Control (RBAC)

Definition & Purpose

Role-based access restricts users based on predefined roles instead of individual permissions. This ensures organized, scalable, and secure access.

Roles in the System

Role Name	Meaning	Permissions / Access Rights
Admin	System Superuser	Full CRUD on all tables, user management, role assignment, audit log access
Doctor	Medical Practitioner	Access patient records, appointments, prescriptions, diagnosis, lab tests; can create/update clinical data
Receptionist	Front-desk Operator	Manage appointments, check-in/out, update patient basic info

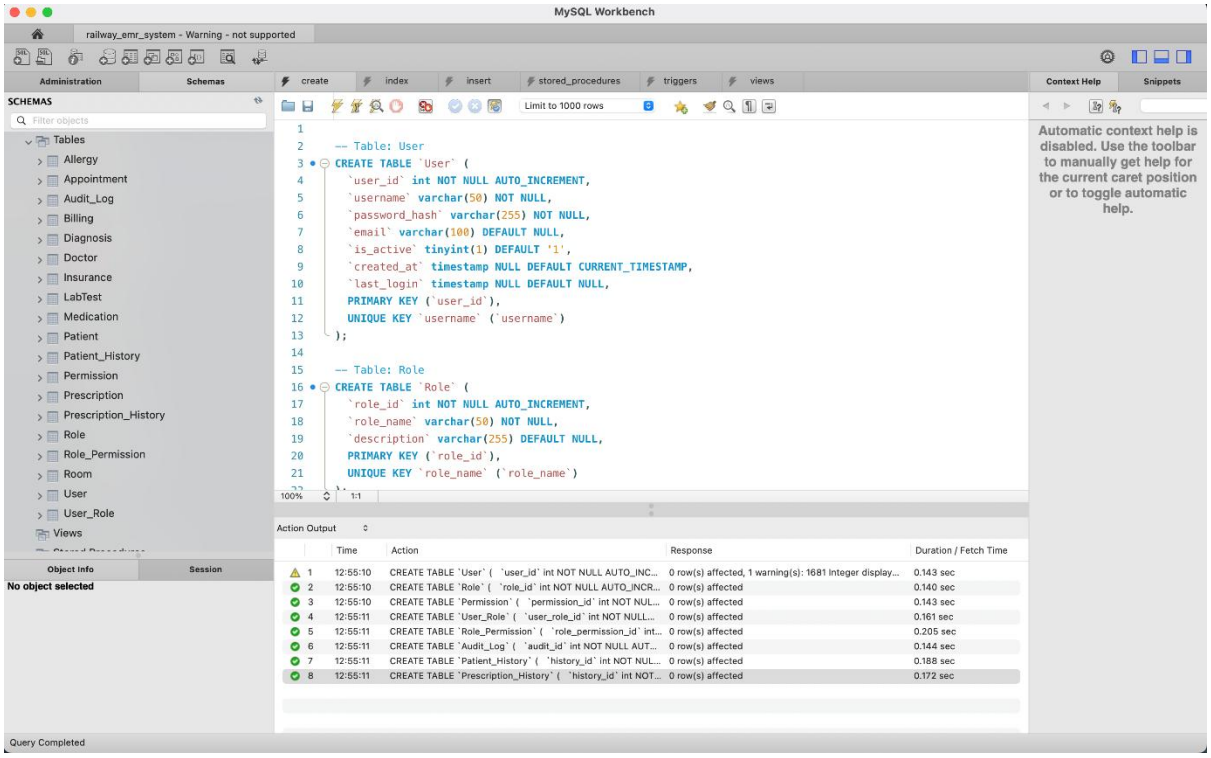
6.6 Audit Trails

Definition & Purpose

Audit trails track changes made to critical database tables to ensure accountability, traceability, and security compliance.

How Audit Trails Work

A trigger logs INSERT, UPDATE, or DELETE actions into audit tables, storing user details, timestamps, and changed values.



7. Tools Used

The development and implementation of this system utilized several tools to ensure efficient design, execution, and testing. MySQL Workbench served as the primary tool for creating the database schema, executing SQL scripts, and visualizing entity relationships. Postman was employed for API development and testing, enabling validation of stored procedures, authentication mechanisms, and role-based access control functionality. For data preparation, Microsoft Excel and Google Sheets were used to format and organize initial datasets before database import. VS Code was used as the integrated development environment for both frontend and backend code development. GitHub provided version control and code repository management, while Vercel enabled seamless deployment and hosting of the application. These tools collectively facilitated streamlined database design, comprehensive testing, efficient development workflows, and thorough documentation throughout the project.

Tool	**Purpose**
MySQL Workbench	Database development and testing
Node.js, Express.js	API backend
React.js	Frontend
GitHub	Version control
VS Code	Application development
Deployment	Vercel

8. GitHub Repository

The complete source code for this Healthcare EMR System is available on GitHub at https://github.com/yaminibuyya27/healthcare-emr-system. The repository contains both frontend backend code, database schemas with stored procedures and triggers, comprehensive documentation.

8.1 Repository Structure

The repository is organized into three main directories:

- **backend/** - Contains Node.js/Express server code, API routes (auth, patients, appointments, prescriptions, admin), and SQL scripts organized by project phases
- **frontend/** - Contains React application with role-based dashboards (Admin, Doctor, Receptionist), login interface, and API client configuration
- **backend/sql_files/** - Complete database setup scripts including schema creation, stored procedures, triggers, views, indexes, and sample data

8.2 Key Features Implemented

The implementation includes comprehensive CRUD operations for patient management, appointment scheduling with calendar views, prescription management with medication database integration, and complete audit logging through database triggers. The system supports role-based access control with distinct interfaces for administrators (audit trail monitoring), receptionists (patient and appointment management), and physicians (appointment viewing and prescription creation). All database operations are secured through stored procedures and views that enforce permission boundaries.

8.3 Live Deployment

The application is deployed and accessible at https://healthcare-emr-system.vercel.app. The deployment uses Vercel for hosting the React frontend, with the backend configured to connect to a cloud-based MySQL database instance. The README.md file in the repository provides detailed setup instructions for local development including environment configuration, database initialization, and running both frontend and backend servers.

8.4 Demo User Credentials

The following test accounts are available for exploring different role-based functionalities:

Role	Username	Password
Administrator	admin	admin123
Receptionist	receptionist_jane	receptionist123
Physician	dr_patel	doctor123

Note: Additional physician accounts (dr_sharma, dr_reddy, dr_kumar, dr_naidu) are also available with the same password "doctor123".